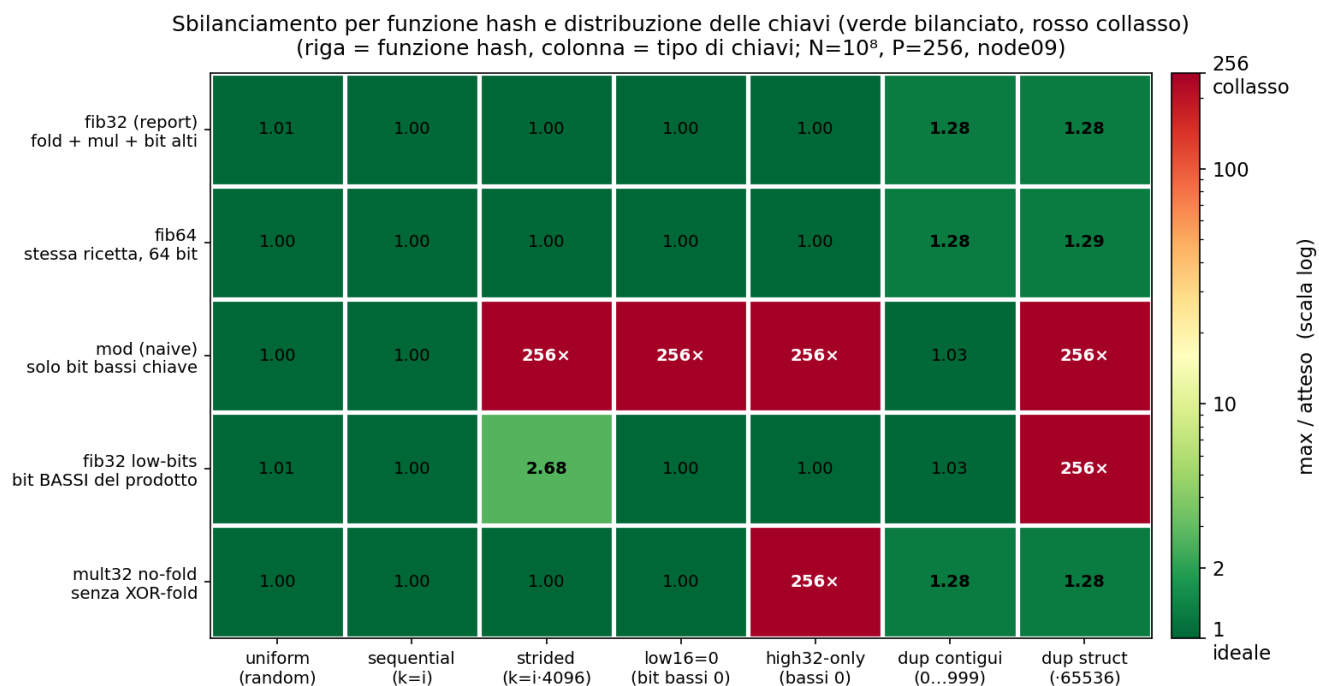


Modulo 1: esperimenti aggiuntivi

Esperimenti a supporto del report del Modulo 1 (vettorizzazione del partition mapping kernel). Per ciascuno il grafico e i punti principali. Misure su node09 (AMD EPYC 7301), lo stesso nodo del report.

Esperimento	Riferimento nel report
Esp. 1: qualità della hash	Sez. 1.1, mapping function
Esp. 2: tetto di banda	Sez. 4.2, regime memory-bound
Esp. 3: grid search dei flag	Sez. 2, auto-vettorizzazione
Esp. 4: 32 vs 64 bit	Sez. 3, AVX2 intrinsics
Esp. 5: rerun CPU e CUDA	Tabella 1 e Sez. 5, CUDA

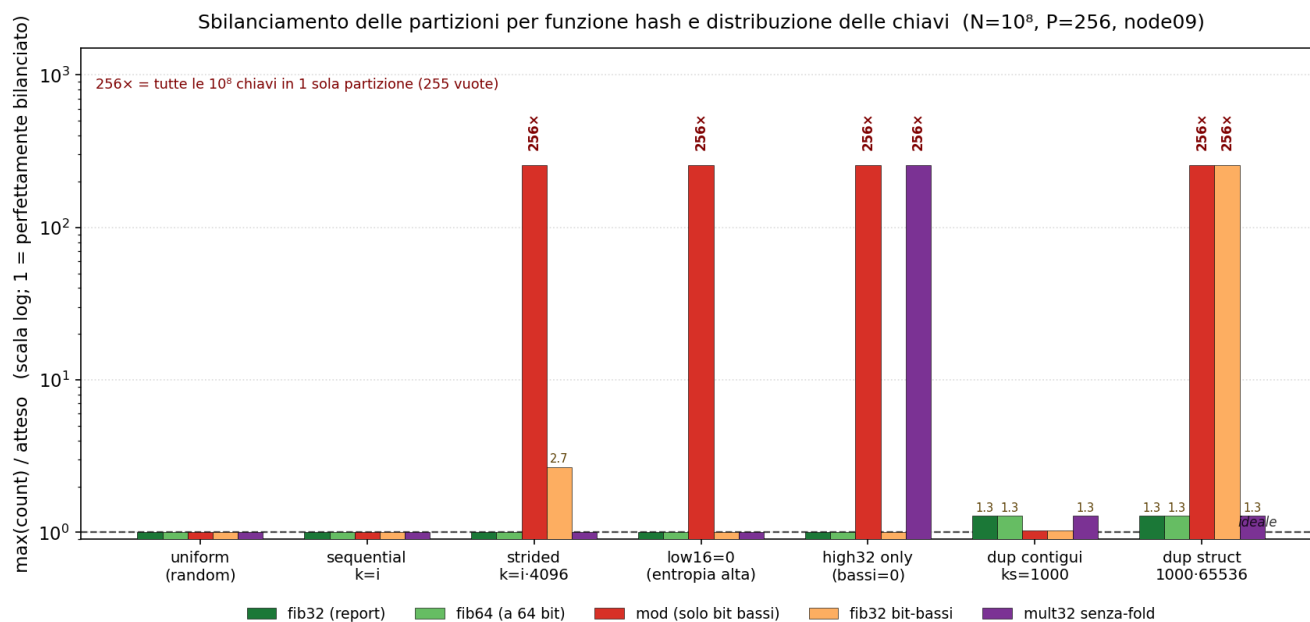
Esp. 1: qualità della hash



Sbilanciamento max/atteso di cinque funzioni hash su sei distribuzioni di chiavi (verde bilanciato, rosso collasso).

- Su chiavi uniformi tutte le funzioni bilanciano (fib32 = 1.01).
- Su chiavi strutturate mod collassa a 256x (tutte le chiavi in una partizione) quando i bit bassi mancano di entropia; le varianti che rimuovono un ingrediente della hash (il fold o i bit alti) falliscono su almeno una distribuzione.
- Solo fib32 e fib64 restano bilanciate su tutte le distribuzioni, con qualità equivalente.
- La scelta di fib32 è motivata dalla robustezza (caso peggiore circa 1.28, nessun collasso), non da una superiorità sistematica: in un join la distribuzione delle chiavi non è nota a priori.

Esp. 1: qualità della hash



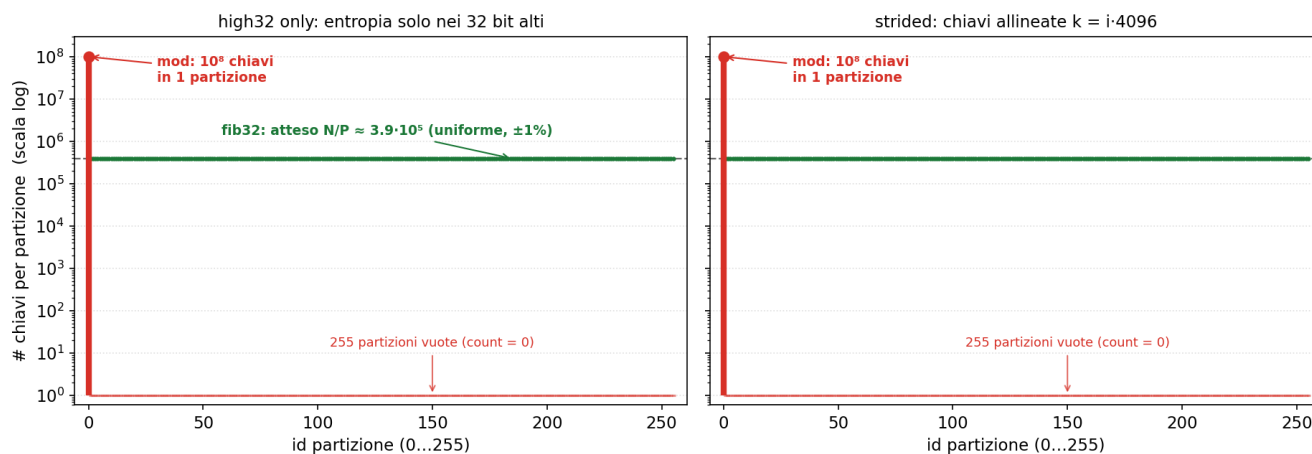
Le stesse misure in barre, in scala logaritmica.

- In scala logaritmica il collasso di mod risulta di ordini di grandezza (fino a 256x), non uno scostamento marginale.
- Un collasso concentra tutte le chiavi in un'unica partizione e compromette la fase successiva.
- fib32 su chiavi uniformi presenta un rapporto max/atteso di 1.005.

Esp. 1: qualità della hash

Occupazione delle 256 partizioni su chiavi strutturate: fib32 bilancia, mod collassa

— fib32 (report): uniforme sulle 256 partizioni — mod (naive): tutto in partizione 0



Occupazione delle 256 partizioni in un caso di collasso.

- fib32 distribuisce le chiavi in modo uniforme sulle 256 partizioni; mod le concentra in poche (partizione 0).
- Il meccanismo dipende dai bit impiegati: mod seleziona i bit bassi della chiave, costanti su chiavi strutturate, mentre fib32 li rimescola con la costante aurea e seleziona i bit alti.
- Lo sbilanciamento è fra partizioni (oggetto del Modulo 1), distinto dal problema di collisione interno alla singola tabella del Modulo 2.

Esp. 2: tetto di banda



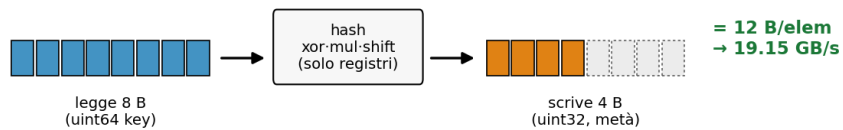
STREAM Scale: il tetto generico, 16.4 GB/s

loop: $a[i] = q \cdot b[i]$ (array di double, 8 byte)



Kernel partition map: il tetto matched, 19.15 GB/s

loop: $\text{part_ids}[i] = \text{hash}(\text{keys}[i])$ (uint64 in, uint32 out)

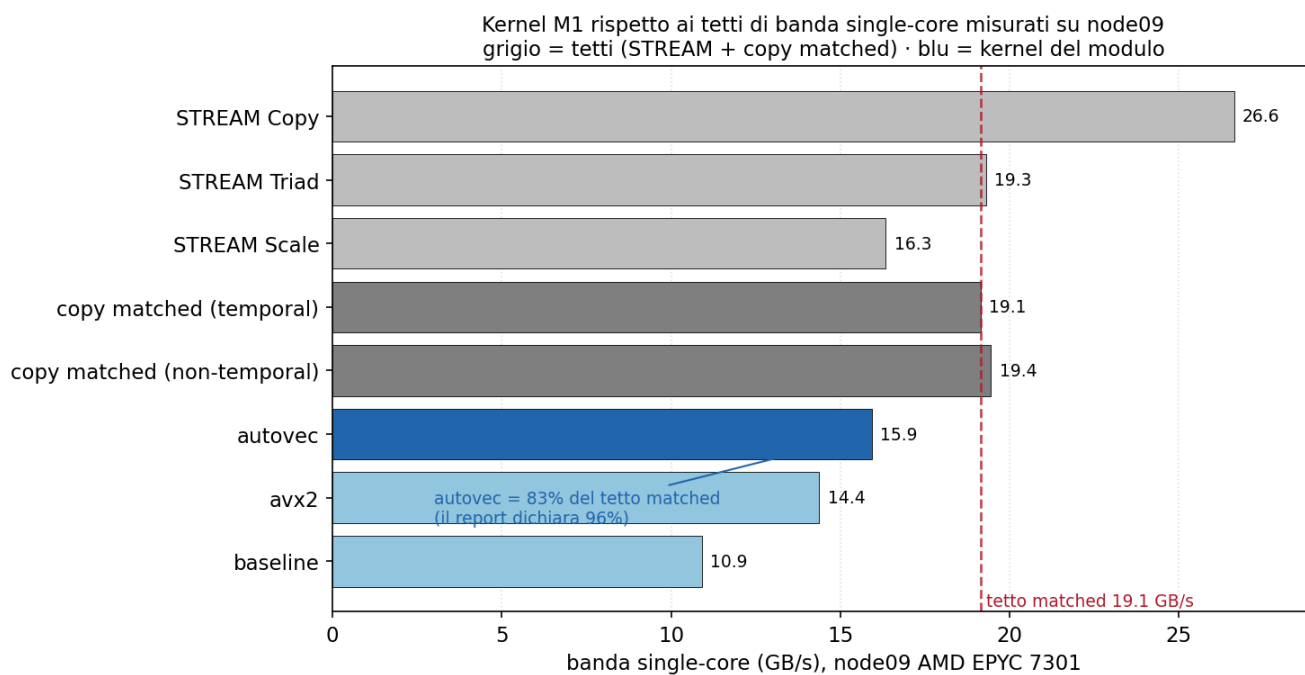


Stessa lettura (8 B), ma il kernel scrive solo 4 B invece di 8 → pattern più leggero → tetto giusto 19, non 16. Confrontare col 16.4 gonfia la saturazione (96% invece di 83%).

Pattern di byte: STREAM Scale legge 8 e scrive 8; il kernel legge 8 e scrive 4.

- Lo STREAM Scale legge 8 B e scrive 8 B per elemento (16.4 GB/s); il kernel legge 8 B (uint64) e scrive 4 B (uint32), con un traffico inferiore.
- Il tetto appropriato per questo pattern è la copia matched (read 8, write 4), pari a 19.15 GB/s, non lo STREAM Scale.
- Il regime resta memory-bound: cambia soltanto il denominatore del rapporto di saturazione.

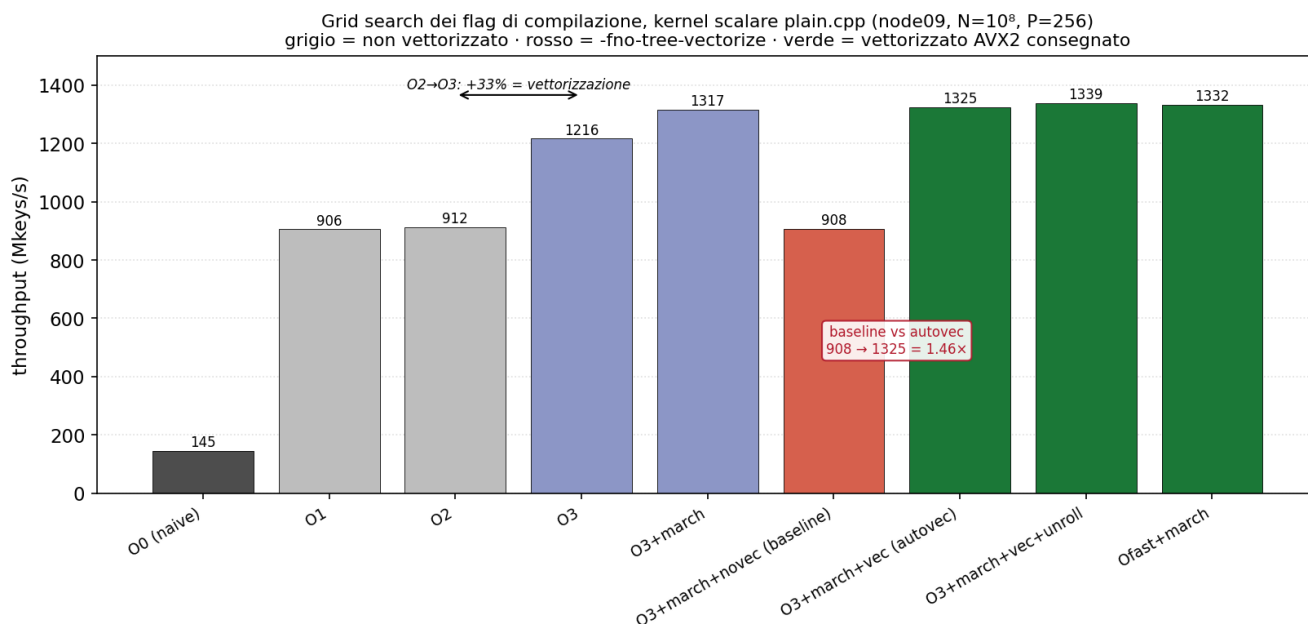
Esp. 2: tetto di banda



Kernel auto-vettorizzato contro i tetti di banda single-core misurati su node09.

- Il kernel auto-vettorizzato raggiunge 15.9 GB/s, rispetto ai tetti misurati sul nodo (Copy 26.6, Triad 19.3, Scale 16.34, matched 19.15).
- Riferito al tetto matched, corrisponde all'83% della banda, non al 96%: quel valore adottava lo STREAM Scale come riferimento.
- Il tetto di 16.4 GB/s citato nel report non è ancorato ad alcuna misura presente nel repository.

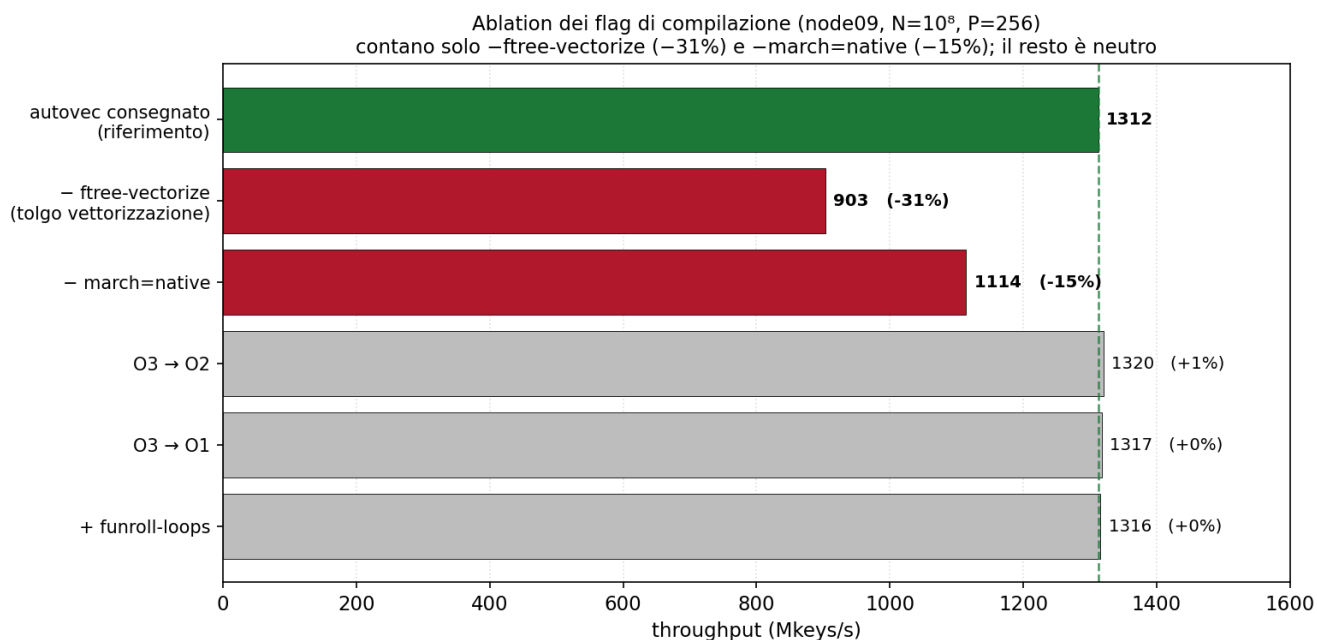
Esp. 3: grid search dei flag



Throughput per configurazione di flag, da O0 naive all'auto-vettorizzato.

- Il throughput cresce da O0 naive (145 Mkeys/s) all'auto-vettorizzato (1325). Gli incrementi principali sono il passaggio da O0 a O1 (register allocation, da 145 a 906) e da O2 a O3 (auto-vettorizzazione, più 33%).
- -Ofast eguaglia O3: su un kernel a interi il fast-math non ha effetto, ed è quindi escluso a ragione.

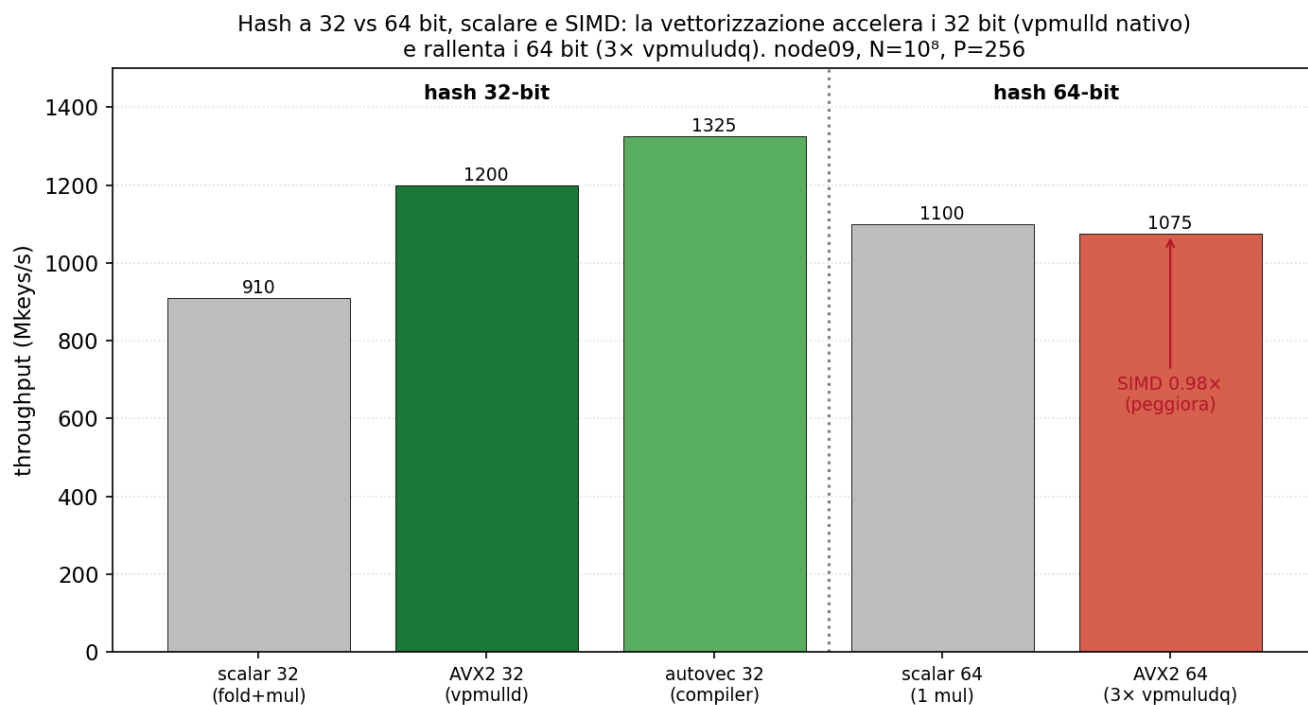
Esp. 3: grid search dei flag



Ablation: rimuovendo un flag alla volta, contano solo vettorizzazione e march.

- La rimozione di -fvec-vectorize costa il 31%, quella di -march=native il 15%; ridurre il livello di ottimizzazione (O2/O1), -funroll-loops e -Ofast non hanno effetto apprezzabile.
- Lo speedup dipende esclusivamente da questi due flag: quelli del Makefile sono l'insieme minimale sufficiente.

Esp. 4: 32 vs 64 bit



- La vettorizzazione accelera la hash a 32 bit ($\text{avx2_32}/\text{scalar32} = 1.32x$) ma rallenta quella a 64 bit ($\text{avx2_64}/\text{scalar64} = 0.98x$).
- In versione scalare la hash a 64 bit è più semplice: scalar64 (1100) supera scalar32 (910).
- I 32 bit sono scelti perché si vettorizzano efficacemente, non per un minor costo scalare; la qualità di partizionamento è equivalente ($\text{fib32} = \text{fib64}$).

Esp. 4: 32 vs 64 bit

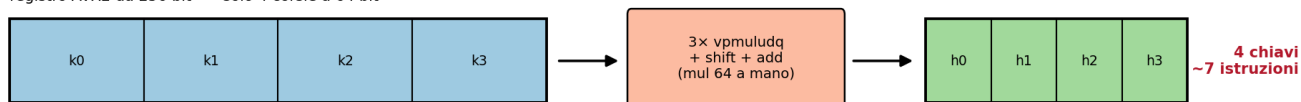
Hash 32 bit: vpmulld (istruzione nativa)

registro AVX2 da 256 bit = 8 corsie a 32 bit



Hash 64 bit: vpmullq non esiste in AVX2

registro AVX2 da 256 bit = solo 4 corsie a 64 bit

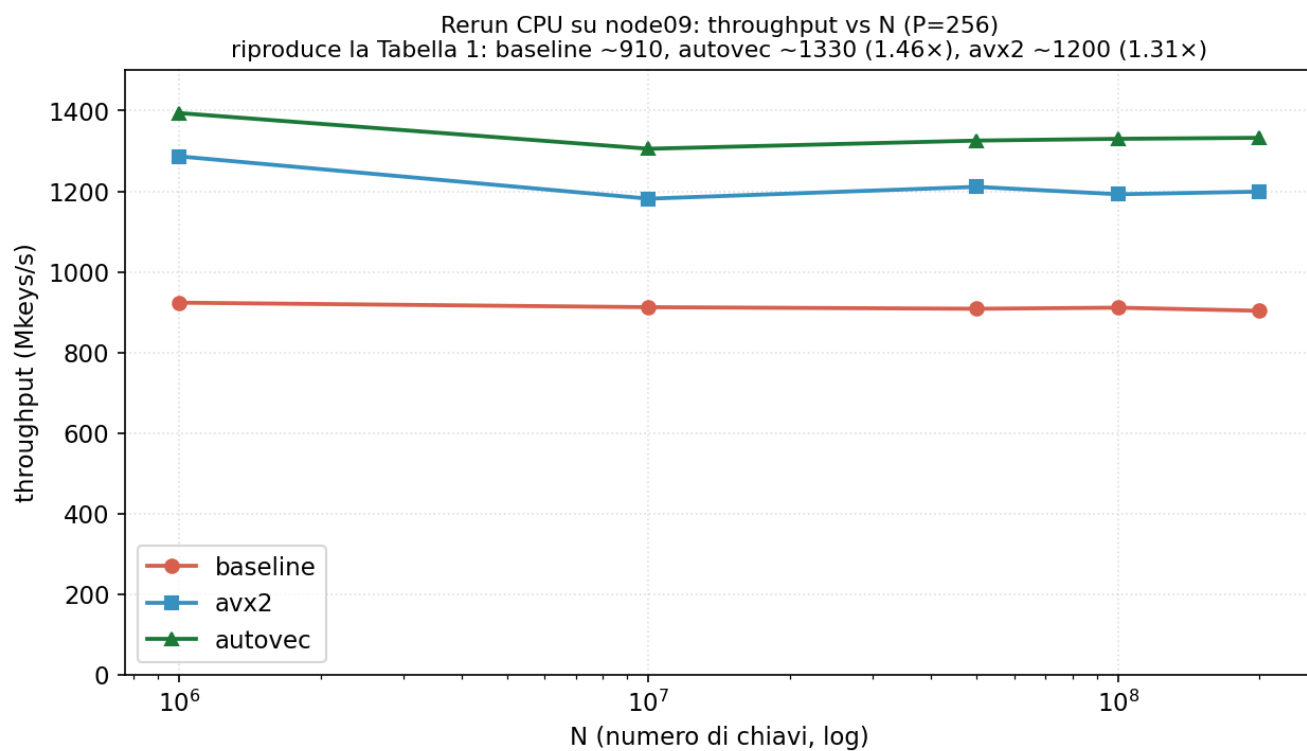


Stesso registro da 256 bit: a 64 bit metà chiavi (4 invece di 8) e ~7x lavoro per mul → la SIMD perde: $avx2_64 = 0.98\times$ (peggiora), $avx2_32 = 1.32\times$ (aiuta).

Registro AVX2 da 256 bit: 8 corsie a 32 bit contro 4 a 64 bit.

- AVX2 dispone della moltiplicazione intera nativa a 32 bit (`vpmulld`, 8 chiavi per istruzione), ma non a 64 bit.
- Una moltiplicazione 64x64 richiede la scomposizione in 3 `vpmulldq`: circa 7 istruzioni invece di una, e metà delle corsie (4 invece di 8).
- La moltiplicazione intera a 64 bit in SIMD (`vpmullq`) è disponibile solo con AVX-512.

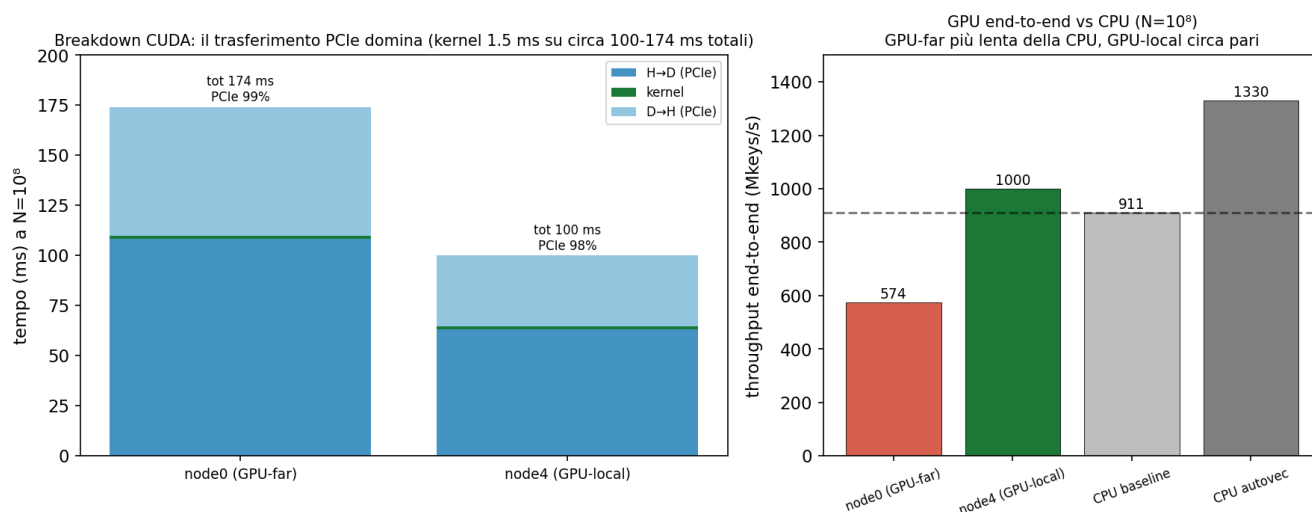
Esp. 5: rerun CPU e CUDA



Throughput delle tre strategie al variare di N; riproduce la Tabella 1.

- La misura riproduce la Tabella 1 (stessi speedup, stessi checksum): a $N=10^8$, $P=256$, baseline circa 910, auto-vettorizzato circa 1330 (1.46x), avx2 circa 1200 (1.31x) Mkeys/s.
- Il throughput è indipendente da N (kernel branchless), e l'ordinamento auto-vettorizzato, avx2, baseline si mantiene su tutte le dimensioni.

Esp. 5: rerun CPU e CUDA



Breakdown CUDA e sensibilità all'affinità NUMA.

- Il kernel raggiunge circa 800 GB/s (81% della banda HBM2), ma il trasferimento PCIe occupa il 98% del tempo end-to-end.
- L'end-to-end dipende dal posizionamento dell'host: sul dominio NUMA della GPU è circa 1000 Mkeys/s (prossimo al 1031 del report), sul socket remoto scende a 574 (banda PCIe dimezzata).
- La conclusione resta valida in ogni caso: per il kernel isolato la GPU non offre vantaggi.